# Why Kafka?

*“Care about what other people think*
*and you will always be their prisoner”*
*- Lao Tzu*



# Outline

1. How to Choose a Message Broker?
2. Why is Kafka performant?
3. Why is Kafka reliable?

# 1. How to Choose a Message Broker?

When choosing a message broker,
What characteristics should be evaluated?
And in which order?

# 1.0. How to Choose a Message Broker?

- **Common Pitfall: Performance First**
- Advision: Consider the **delivery guarantee and functionality first**
- Evaluating message broker on characteristics:
  - **Delivery Guarantee**
  - **Functionality**
  - **Performance**
  - **Scalability**
  - **Ecosystem**

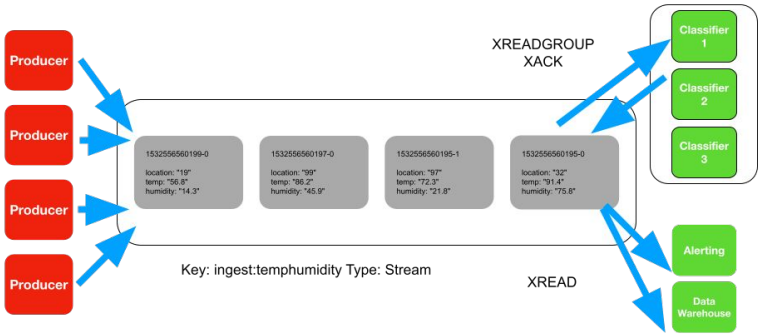
# 1.1. Database-Backed Queue

- Pros:
  - **Leveraging existing infrastructure**
  - **Consistency (exactly-once)**
- Cons
  - **Low performance** (bottleneck: database)
  - Complex to handle errors
  - **Poor features**
- Case Study: [Starbucks uses a database table as a queue]

| order_no | status | updated_at |
|---|---|---|
| 34 | DONE | 12:11:00 |
| 35 | IN_PROCESSING | 12:11:02 |
| 36 | IN_PROCESSING | 12:11:03 |
| 37 | NEW | 12:11:08 |

# 1.2. Redis

- Redis works as message brokers with 3 features:
  - List (at-most-once)
  - Pub/Sub (at-most-once)
  - Stream (at-least-once)




- Pros
  - Leveraging existing infrastructure
  - **Low Latency**
  - **Simplicity**
- Cons
  - **Durability concerns**
  - **Resource intensiveness**
  - Operational overhead
  - **Poor features for a message broker**

# 1.3. RabbitMQ

- Pros:
  - **Flexible Routing**
  - RabbitMQ supports at-least-once
  - Cool features (Delay, Priority Queue, RPC, …)
  - **Low Latency**
  - Wide Support
- Cons
  - **Deleting messages on ACK**
  - **Medium performance**
  - Resource intensiveness
  - **Ecosystem**

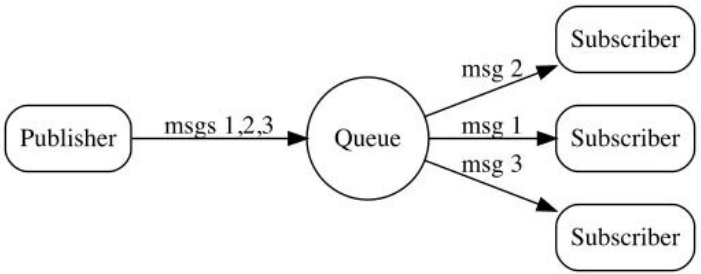
| | RabbitMQ | kafka |
|---|---|---|
| Written in | Erlang | Scala, Java |
| Protocol | AMQP | binary protocol over TCP |
| Client API | Java, Ruby, JavaScript, Go, C, Swift, Spring, Elixir, PHP, and .NET | Java, Ruby, Python, Node.js |
| Flexible Routing Rules | support in Exchange component | No |
| Message Consumption | push | pull |
| Message Priority | ✓ | No |
| Message Ordering | ordered in a queue | ordered in a topic |
| Message Deletion | delete on ACK | delete on retention period expires |
| Security | manage access via admin tools | TLS, JAAS |
| Scalability | RabbitMQ consistent hash exchange | Add more partitions to a topic |
| Fault tolerance | ✓ | ✓ |
| Message pileup back pressure | cannot handle very well | designed to hold messages |
| Performance | tens of thousands/sec | millions/sec |
| Ecosystem | not as good as Kafka | Well supported in big data and stream computing |



Credit: bytebytego

# 1.4. NATs

- Pros:
  - **Higher performance than Kafka**
  - **Flexible Routing**
  - Less operational overhead
- Cons:
  - Random load balancing → **ordering issue**
  - **At-most-once delivery**
- Read more



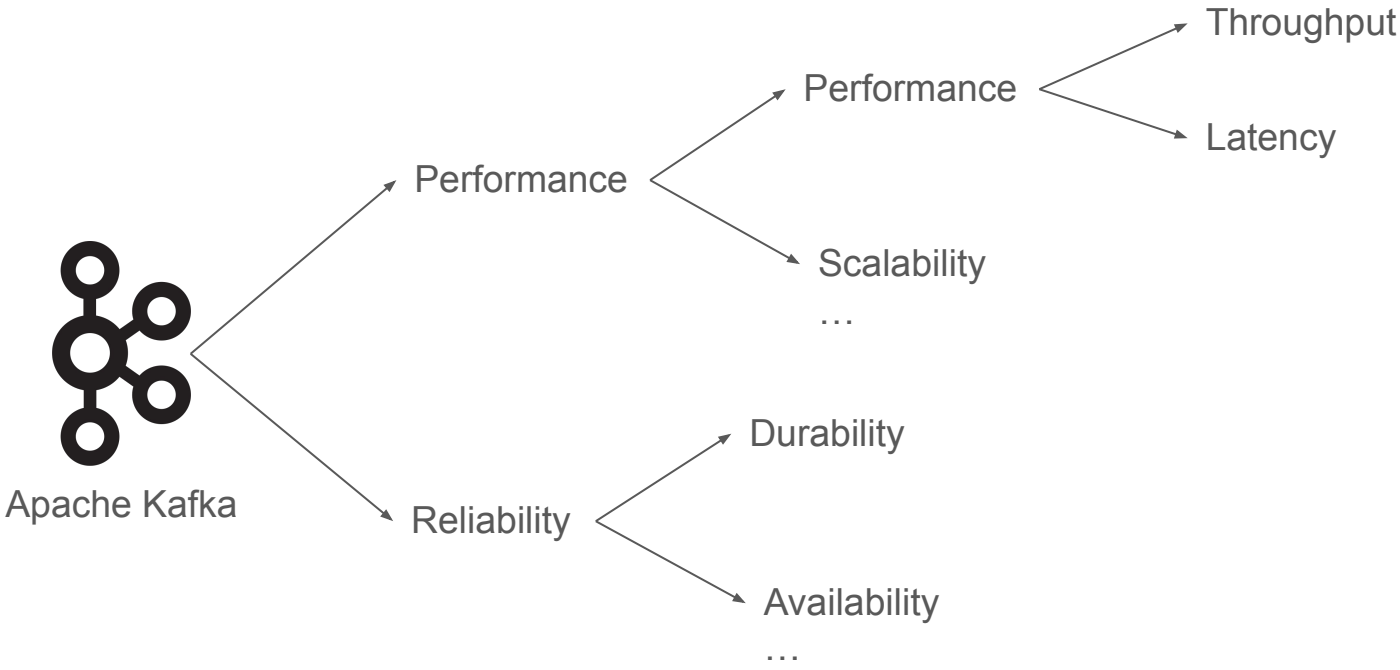
# 1.5. Pulsar

- Pulsar is quite similar to Kafka
- Pros:
  - High Performance (< Kafka)
  - Scalability (< Kafka)
  - **Cool features**
    - Queuing
    - Multi-tenancy
- Cons:
  - Operational Overhead
  - **Ecosystem**
- Read more



# 1.6. How to Choose a Message Broker?

- Evaluating a message broker on characteristics:
  - Delivery Guarantee
  - Functionality
  - Performance
  - Scalability
  - Ecosystem
- Kafka is popular but not a silver bullet.
- Don’t trust opinionated benchmarks from others!


Throughput
Performance
Latency
Performance
Scalability
…
Apache Kafka
Durability
Reliability
Availability
…

# 2. Why is Kafka performant?

# 2.0. Why is Kafka performant?

- Correct misunderstanding: **Actually Apache Kafka is not fast, is optimized for throughput** at the expense of latency, while preserving other desirable qualities, such as durability, strict record order, and at-least-once delivery semantics.
- A lot of design decisions:
  - Broker Side
  - Client Side
  - Stream Parallelism



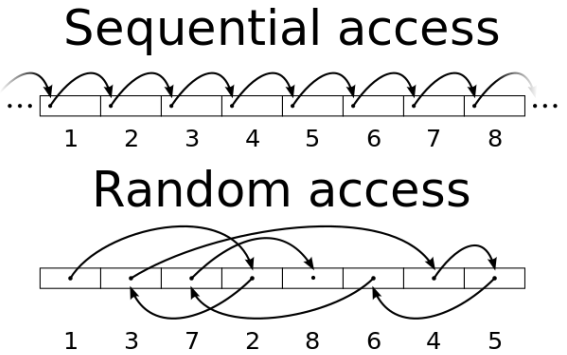
# 2.1. Broker Side

# Are disk accesses slow or fast?

# 2.1.1. Sequential I/O

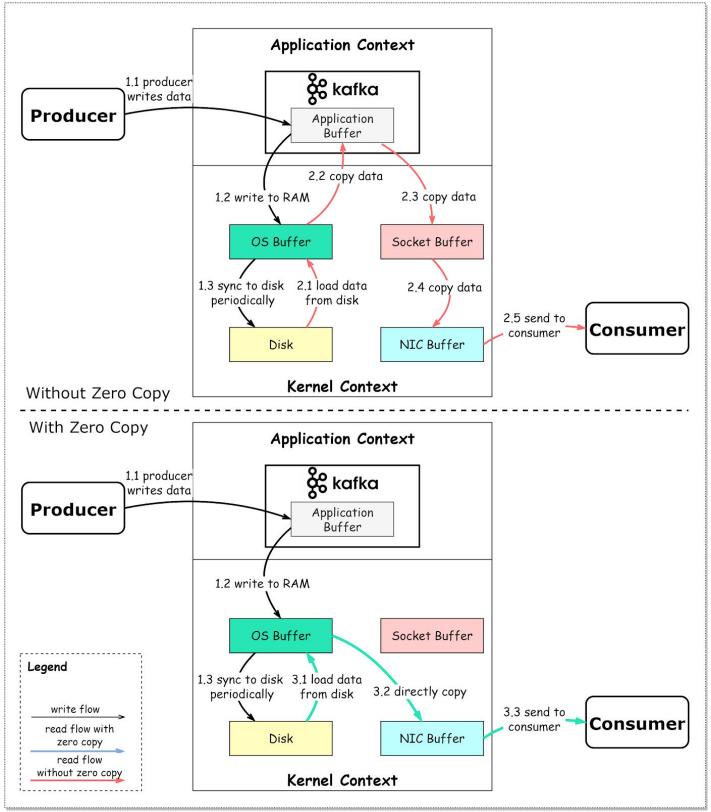
- **A wide misconception: disks are slow**
- **Sequential access is much faster than random access** (x3 - 4), which jumps to many different locations on disk.
- Understanding I/O: Random vs Sequential | flashdba
- Kafka utilizes a segmented, **append-only log**
- **Sequential I/O for both reads and writes**





# 2.1.2. Zero-Copy

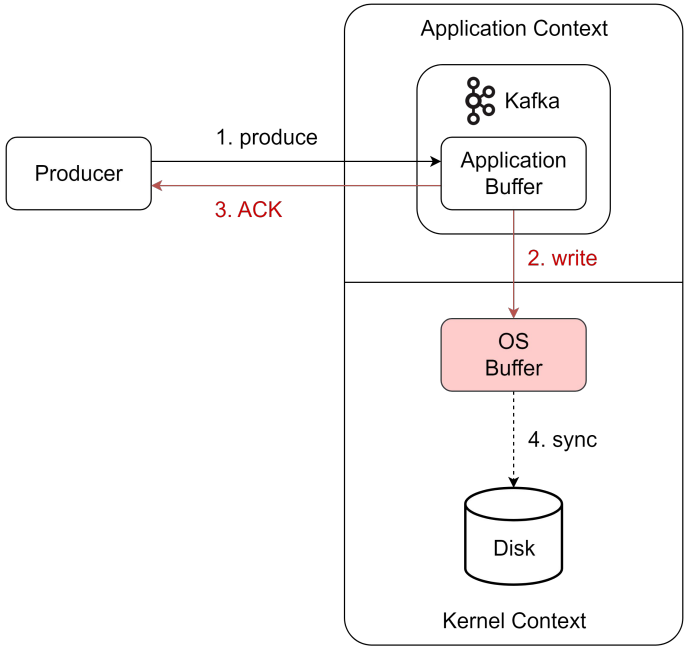
- **Context switching and CPU copy are costly**
- Zero-Copy mechanism reduces context switches and CPU copy significantly
- Zero-Copy is disable when
  - **SSL connections** to clients
  - Brokers execute compression




Credit: bytebytego

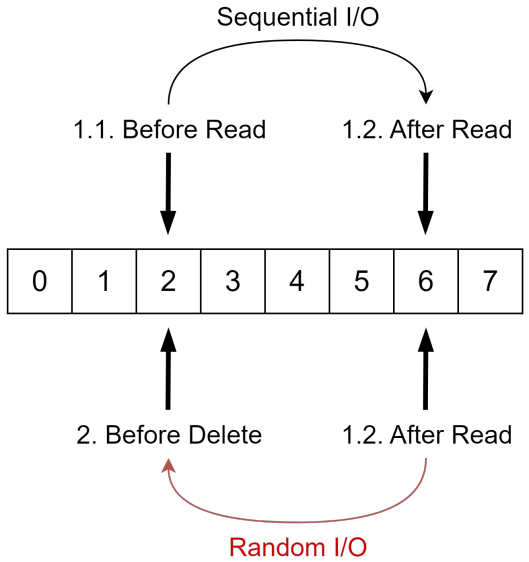
# 2.1.3. Unflushed Buffered Writes

- Kafka doesn't actually call fsync when writing to the disk before acknowledging the write
- Broker **return ACK after record is written to the I/O buffer, not after fsync to the disk.**
- This is unsafe → in-sync replicas to make Kafka durable



# 2.1.4. Cheap Consumer

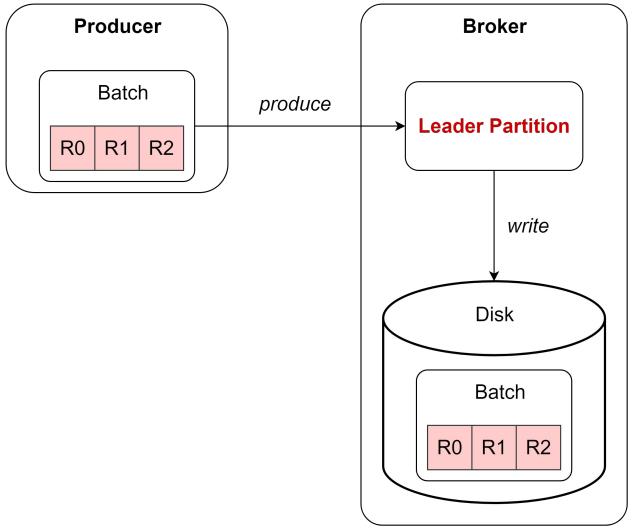
- **Traditional MQ-style brokers which remove messages when consuming**
  **→ the penalty of random I/O**
- **Kafka doesn't remove messages after they are consumed**
- It independently tracks offsets at each consumer group level (using a topic __consumer_offsets). This is fast



# 2.2. Client Side

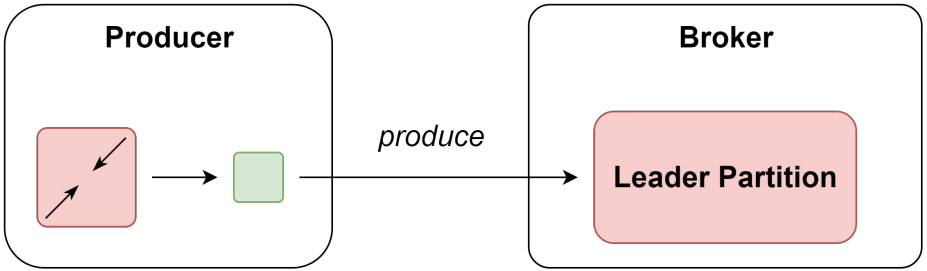
# 2.2.1. Batching

- Problem:
  - A wide misconception: the bottleneck with Kafka is the disk
  - Usually **the bottleneck is network**.
- Solution:
  - **Batching of records improves network bandwidth and throughput**



# 2.2.2. Compression

- Problem: Size of a batch can be big
- Solution: **Batch Compression**
  - Improve network bandwidth
  - Improve storage space, disk I/O utilization



# 2.2.3. Client-Side Optimizations

- Problem:
  - Generally, client is simpler than the server significantly
  - **Server/Broker handles a bulk of the load**
- Solution:
  - **Client performs a significant amount of work**
  - Example: batching records, compressing, hashing keys, checksumming, refreshing the metadata of cluster, …

# 2.3. Stream Parallelism

# 2.3. Stream Parallelism

- **Partitioning mechanism allows for:**
  - **Parallelism (throughput)**
  - **The horizontal scalability**
  - Availability, Durability
- **Producers can publish to multiple partitions concurrently** and partitions handle in parallel
- **Consumers (in a group) subscribe the topic and process messages in parallel**
- **Load-balancing** mechanism: distributing partition assignments approximately evenly among the individual consumer instances within the group.

